

Evaluation, Observability + Docker

You cannot improve what you cannot measure.

You cannot ship what you cannot containerise.

Wednesday | Week 4, Day 3

Today's Agenda

Evaluate → Observe → Ship: a production loop for measurable, portable AI systems.

PHASE 01

Evaluate

26m

- 01 Evaluation frameworks
- 02 LLM-as-Judge
- 03 AI product metrics
- 04 Structured logging

PHASE 02

Observe

17m

- 05 Observability vs monitoring
- 06 LangFuse vs LangSmith
- 07 OpenTelemetry for AI

PHASE 03

Ship

15m

- 08 Docker fundamentals
- 09 GitHub Actions CI pipeline

Monday → Today Monday hardened the code; today we make it measurable, observable and portable.

PHASE 1

Evaluation

Frameworks · LLM-as-Judge · AI Product Metrics

Measure what matters before you optimise what moves.

Evaluation Frameworks

Which to use when — three layers, three different questions.

LEVEL 01

RAGAS

Retrieval quality

Precision, recall,
faithfulness

WHEN TO USE

Your AI uses retrieval

LEVEL 02

LangSmith Evals

LLM response quality

Agent behaviour,
prompt regression

WHEN TO USE

Wire evals into CI/CD

LEVEL 03

Custom Test Suite

Business logic

Your edge cases
and policies

WHEN TO USE

Always — your domain
is unique

Use all three. RAGAS = retrieval · LangSmith = LLM quality · Custom tests = business logic

Next step Wire evaluation into GitHub Actions later today.

RAGAS: Quick Recap + Where It Fits in Week 4

● ● ●
Use retrieval metrics as a live quality signal, not a one-time notebook

≥ 0.80	Context Precision retrieved chunks are relevant
≥ 0.80	Context Recall no important chunks missed
≥ 0.85	Answer Relevancy answer addresses the question
≥ 0.90	Faithfulness no hallucination

```
@app.post("/evaluate")
async def evaluate(question, answer,
contexts):
    dataset = Dataset.from_dict({
        "question": [question],
        "answer": [answer],
        "contexts": [contexts]
    })
    scores = evaluate(dataset,
        metrics=[faithfulness,
        answer_relevancy])
    return scores
```

Why expose /evaluate?

QA teams can test documents without touching code; monitoring can call it to check live response quality.

LLM-as-Judge: Evaluate at Scale

Automate quality review when human labelling becomes the bottleneck

Human review problem

Testing 10,000 responses
manually
can take weeks.

LLM-as-judge

One API call scores one
response.
The same pattern scales to
millions.



Cost reality: judging 10,000 responses with Claude Haiku is roughly a few dollars — far cheaper than human labelling.

LLM-as-Judge: Prompt Design

The judge prompt is the product spec for evaluation

```
SYSTEM: You are an expert evaluator.  
Score 0-10 using:  
- Accuracy  
- Completeness  
- Faithfulness to provided context  
  
Return ONLY JSON:  
{ "score": 0-10, "reason": "one sentence", "pass":  
true/false }  
  
INPUTS: Question, Context, AI response
```

- 01 Define criteria**
Never leave “quality” undefined.
- 02 Require JSON**
Free-text scoring is unreliable.
- 03 Include context**
Judge only what the AI could see.
- 04 Use a different model**
Avoid same-model bias.
- 05 Calibrate first**
Check 10–20 human-scored examples.

Structured output → metric.

AI Product Metrics: What to Track

Four numbers reveal most product failures

01

Latency p95

Slow-tail user experience

< 3s chat

02

Cost per query

API spend per request

budgeted

03

Hallucination rate

unsupported claims

< 5%

04

User satisfaction

was the need met?

> 80% positive

Why p95 catches problems

Averages hide the 5% of users waiting 8+ seconds.

Set alerts on:

- Latency p95 > 5s
- Cost/query > threshold
- Hallucination rate > 5%
- Error rate > 1%

Structured Logging: What to Log and Why



Logs should be searchable, alertable and dashboard-ready

Unstructured log

```
INFO: Got request
INFO: Calling LLM
INFO: Done
```

Python: use structlog or standard logging with a JSON formatter.

Structured log

```
{
  "request_id": "req_a1b2c3",
  "endpoint": "/chat",
  "model": "claude-sonnet-4-6",
  "input_tokens": 842,
  "output_tokens": 215,
  "latency_ms": 1243,
  "cost_usd": 0.0058,
  "faithfulness_score": 0.94
}
```

■ timestamp

■ latency_ms

■ request_id

■ error

■ model

■ cost_usd

■ input/output tokens

■ faithfulness_score

PHASE 2

Observability

Monitoring · LangFuse · LangSmith · OpenTelemetry

See the inside of your AI system one request at a time.

Observability vs Monitoring

Monitoring catches known regressions; observability explains unknown failures

Monitoring

You know what can go wrong.
You watch for thresholds.

Alerts: latency > 3s, errors > 1%
Dashboards: known metrics over time

Observability

You can understand anything that went wrong — even surprises.

Example: traces reveal embedding calls spiked after a large PDF was chunked incorrectly.

Three pillars for AI systems

Logs

what happened

Metrics

how it performs

Traces

the request journey

LangFuse vs LangSmith

Both provide traces, evals, prompt management and cost tracking

LangSmith

Creator	LangChain
Hosting	Cloud only
Framework	LangChain-native
Works best	LangChain / LangGraph
Data	LangChain-managed
Setup	1 env variable
Best for	LangChain-heavy projects

LangFuse

Creator	Open-source community
Hosting	Self-hosted or cloud
Framework	Framework-agnostic
Works best	Any model/framework
Data	Your servers if self-hosted
Setup	Docker compose or cloud
Best for	Privacy, multi-framework teams

Rule Data privacy → LangFuse self-hosted. Individual project or LangChain shop → LangSmith.

OpenTelemetry for AI

Vendor-neutral traces, metrics and logs for every LLM call

What it is

An open-source CNCF standard for collecting traces, metrics and logs from any application.

January 2025 milestone

AI / LLM semantic conventions made every model call a standardised traceable span.

```
from opentelemetry import trace
# one setup → chosen backend
```

```
span: llm.generate
model: claude-sonnet-4-6
input_tokens: 842
output_tokens: 215
latency_ms: 1243
cost_usd: 0.0058
error: null
```

Instrument once → switch backends later

LangFuse

Datadog

Grafana

New Relic

PHASE 3

Docker + CI/CD

Containerise · Dockerfile · GitHub Actions

Package the app, test every push, deploy with confidence.

Docker: Why Every AI App Needs It

Stop “works on my machine” before it reaches production

Without Docker

Python versions differ. Libraries drift.
CUDA drivers and environment variables
go missing.

Docker solves this

Package code, dependencies and runtime
into one image that runs anywhere.

- **Reproducibility** same environment everywhere
- **Portability** AWS/GCP/Azure/HF Spaces

- **Isolation** app, database, vector store separate
- **Scaling** spin up more copies automatically

Dockerfile: Structure and Explanation

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
RUN useradd -m appuser && chown -R appuser /app
USER appuser
HEALTHCHECK CMD curl -f /health || exit 1
EXPOSE 8000
CMD ["uvicorn", "main:app"]
```

- 1 Base** Python + OS
- 2 Workdir** inside container
- 3 Cache** copy requirements first
- 4 App code** copy the project
- 5 Security** non-root user
- 6 Health** container liveness
- 7 Start** expose + run uvicorn

Every line has a reason. Know why it is there.

Docker: Build and Run Commands

Build once, run everywhere and never bake secrets into the image

Build image

```
docker build -t ai-writing-coach:v1 .  
# -t = tag, . = current directory
```

Test endpoint

```
curl -X POST http://localhost:8000/chat \  
-H "Content-Type: application/json" \  
-d '{"message": "Fix my grammar"}'
```

Run container

```
docker run -p 8000:8000 \  
-e OPENAI_API_KEY=$OPENAI_API_KEY \  
-e ANTHROPIC_API_KEY=$ANTHROPIC_API_KEY \  
ai-writing-coach:v1
```

.dockerignore is required

```
.env  
*.pyc  
__pycache__  
notebooks/  
.git
```

Common mistake:
forgetting .dockerignore
can push API keys into
the image.

GitHub Actions: CI Pipeline

Every push runs automated checks before code reaches production

- 01 checkout** get the code
- 02 setup-python** Python 3.11
- 03 install** requirements.txt
- 04 lint** ruff check .
- 05 test** pytest tests/ -v
- 06 build** docker build

CI = Continuous Integration: catch bugs before users do.

```
name: AI App CI
on: [push, pull_request]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v4
      - run: ruff check .
      - run: pytest tests/ -v
      - run: docker build -t ai-app:test .
```

GitHub Actions: What Each Step Does

Three checks, two to three minutes, immediate feedback

Lint

ruff

syntax errors, unused imports, style issues

Tests

pytest

logic errors, broken endpoints, regressions

Docker build

Docker

missing files, bad dependencies, invalid Dockerfile

How secrets work

- 01** Repo → Settings → Secrets and variables → Actions
- 02** Add OPENAI_API_KEY and ANTHROPIC_API_KEY
- 03** Use as `${{ secrets.OPENAI_API_KEY }}`
- 04** GitHub masks secrets in logs automatically

This is what “CI/CD” means on job descriptions.

Result

Every git push → checks run. Fail fast or deploy with confidence.

Full Production Stack: Everything Together

The complete path from commit to observed, evaluated production AI

Developer pushes code

GitHub Actions CI

lint → test → build Docker image

Docker registry

image stored after checks pass

Production server

pulls latest image

FastAPI in Docker

auth, rate limit, validation, prompt-injection scan, async LLM, logging, health

Observability layer

LangFuse / LangSmith traces, OpenTelemetry spans, alerts

Evaluation layer

RAGAS endpoint, LLM-as-judge, custom CI tests

This is a production AI system. You built every layer of this over four weeks.



Key Takeaways: Wednesday

Evaluation, observability and shipping are one production loop

Evaluate

- RAGAS checks retrieval
- LangSmith checks LLM quality
- Custom tests prove business logic

Observe

- Track latency p95, cost, hallucinations, satisfaction
- Log request ID, model, tokens, latency, cost, errors, quality
- Monitoring catches known problems; observability explains unknowns

Judge

- LLM-as-judge scales review
- Define criteria and require JSON
- Calibrate with human examples

Ship

- LangSmith = LangChain cloud; LangFuse = self-hosted option
- OpenTelemetry: instrument once, switch backend
- Docker + GitHub Actions: same environment, every push checked

Thursday lab: add /evaluate, structured logging, hardened Dockerfile and GitHub Actions workflow; deploy to Hugging Face Spaces.

Appendix: Quick Reference Card

Architecture selection, production checklist and shipping commands

Architecture selection

Dynamic short task	ReAct
Known multi-step	Plan-and-Execute
Multi-hop retrieval	Agentic RAG
Specialised tasks	Supervisor pattern
Parallel tasks	Peer collaboration

FastAPI production checklist

- async / await LLM calls
- Pydantic validation
- API keys in .env only
- rate limiting + auth
- timeouts + fallbacks
- streaming endpoints
- structured JSON logs

Shipping commands

Docker

```
build -t app:v1 .  
run -p 8000:8000  
env KEY=val  
ps / logs <id>
```

GitHub Actions

```
push → checkout → setup-  
python → lint → test →  
docker build
```

Use this card when building your next production AI service.

THANK YOU

iCodeGuru × DigiTech Transformation
